

TIMESTAMP BASED CONCURRENCY CONTROL

- Each transaction T_i is issued a timestamp $TS(T_i)$ when it enters the system.
 - Each transaction has a unique timestamp
 - Newer transactions have timestamps strictly greater than earlier ones
 - Timestamp could be based on a logical counter
 - Real time may not be unique
 - Can use (wall-clock time, logical counter) to ensure
- Timestamp-based protocols manage concurrent execution such that time-stamp order = serializability order
- Several alternative protocols based on timestamps

With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by $TS(T_i)$. This timestamp is assigned by the database system before the transaction T_i starts execution. If a transaction T_i has been assigned timestamp $TS(T_i)$, and a new transaction T_j enters the system, then $TS(T_i) < TS(T_j)$. There are two simple methods for implementing this scheme:

1. Use the value of the **system clock** as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
2. Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

TS:

10	11	12
T_1	T_2	T_3
↑		↑

10	30	20
T_1	T_2	T_3
$T_1 \rightarrow T_3 \rightarrow T_2$		
10	20	30

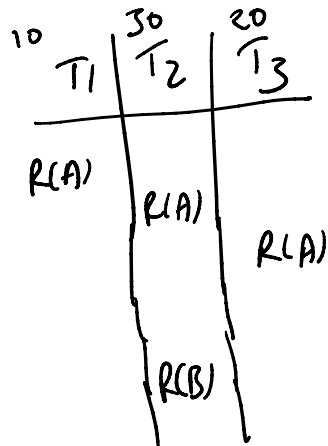
Concurrency control techniques based on timestamp ordering do not use locks; hence, deadlocks cannot occur.

The idea for this scheme is to enforce the equivalent serial order on the transactions based on their timestamps. A schedule in which the transactions participate is then serializable, and the only equivalent serial schedule permitted has the transactions in order of their timestamp values. This is called **timestamp ordering (TO)**. Notice how this differs from 2PL, where a schedule is serializable by being equivalent to some serial schedule allowed by the locking protocols. In timestamp ordering, however, the schedule is equivalent to the particular serial order corresponding to the order of the transaction timestamps. The algorithm allows interleaving of transaction operations, but it must ensure that for each pair of conflicting operations in the schedule, the order in which the item is accessed must follow the timestamp order. To do this, the algorithm associates with each database item X two timestamp (TS) values:

1. **read_TS(X)**. The **read timestamp** of item X is the largest timestamp among all the timestamps of transactions that have successfully read item X —that is, $\text{read_TS}(X) = \text{TS}(T)$, where T is the *youngest* transaction that has read X successfully.
2. **write_TS(X)**. The **write timestamp** of item X is the largest of all the timestamps of transactions that have successfully written item X —that is, $\text{write_TS}(X) = \text{TS}(T)$, where T is the *youngest* transaction that has written X successfully. Based on the algorithm, T will also be the last transaction to write item X , as we shall see.

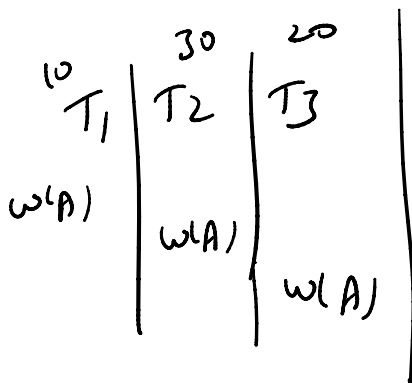
The timestamp ordering (TSO) protocol

- Maintains for each data Q two timestamp values:
 - **W-timestamp(Q)** is the largest time-stamp of any transaction that executed **write(Q)** successfully.
 - **R-timestamp(Q)** is the largest time-stamp of any transaction that executed **read(Q)** successfully.
- Imposes rules on read and write operations to ensure that
 - any conflicting operations are executed in timestamp order
 - out of order operations cause transaction rollback



$$RTS(A) = \cancel{10} \cancel{30}$$

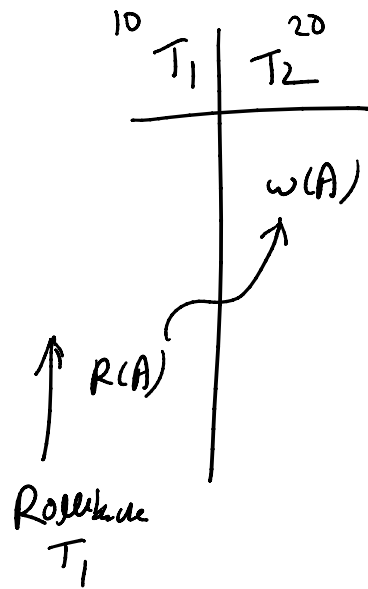
$$RTS(B) = \cancel{20} \cancel{30}$$



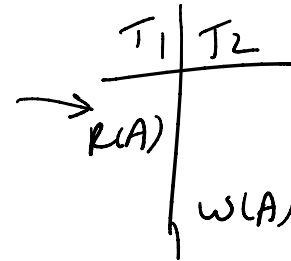
$$WTS(A) = \cancel{10} \cancel{20} \cancel{30}$$

Basic Timestamp Ordering (BTO)

Case I -



$T_1 \rightarrow T_2$

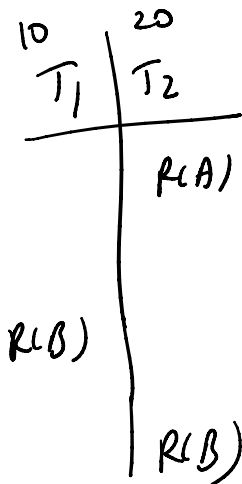


$T_1: R_1(A)$

$WTS(A) > TS(T_1)$

$20 > 10$

$T_1 \uparrow$



$R(B)$

$R(B)$

S

$T_1 \rightarrow T_2$



$R(B)$

$R(A)$

$R(B)$

S'

$S \equiv S'$

Case III

¹⁰	²⁰
T_1	T_2
	$w(A)$
$w(A)$	

↑
Role

$T_1 \rightarrow T_2$

$T_1: w_1(A)$

$WTS(A) > TS(T_1)$

$20 > 10$

Case IV

¹⁰	²⁰
T_1	T_2
	$R(A)$
$R(A)$	

S

$T_1 \rightarrow T_2$

T_1	T_2
$R(A)$	
	$R(A)$

S'

$S \equiv S'$

Basic Timestamp Ordering (TO). Whenever some transaction T tries to issue a $\text{read_item}(X)$ or a $\text{write_item}(X)$ operation, the **basic TO** algorithm compares the timestamp of T with $\text{read_TS}(X)$ and $\text{write_TS}(X)$ to ensure that the timestamp order of transaction execution is not violated. If this order is violated, then transaction T is aborted and resubmitted to the system as a new transaction with a *new timestamp*. If T is aborted and rolled back, any transaction T_1 that may have used a value written by T must also be rolled back. Similarly, any transaction T_2 that may have used a value written by T_1 must also be rolled back, and so on. This effect is known as **cascading rollback** and is one of the problems associated with basic TO, since the schedules produced are not guaranteed to be recoverable. An *additional protocol* must be enforced to ensure that the schedules are recoverable, cascadeless, or strict. We first describe the basic TO algorithm here. The concurrency control algorithm must check whether conflicting operations violate the timestamp ordering in the following two cases:

1. Whenever a transaction T issues a $\text{write_item}(X)$ operation, the following check is performed:
 - a. If $\text{read_TS}(X) > \text{TS}(T)$ or if $\text{write_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation. This should be done because some younger transaction with a timestamp greater than $\text{TS}(T)$ —and hence after T in the timestamp ordering—has already read or written the value of item X before T had a chance to write X , thus violating the timestamp ordering.
 - b. If the condition in part (a) does not occur, then execute the $\text{write_item}(X)$ operation of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.
2. Whenever a transaction T issues a $\text{read_item}(X)$ operation, the following check is performed:
 - a. If $\text{write_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation. This should be done because some younger transaction with timestamp greater than $\text{TS}(T)$ —and hence after T in the timestamp ordering—has already written the value of item X before T had a chance to read X .
 - b. If $\text{write_TS}(X) \leq \text{TS}(T)$, then execute the $\text{read_item}(X)$ operation of T and set $\text{read_TS}(X)$ to the larger of $\text{TS}(T)$ and the current $\text{read_TS}(X)$.

$$\text{write_TS}(X) \leq \text{TS}(T)$$

$$\text{write_TS}(X) = \text{TS}(T)$$

$$\text{read_TS}(X) = \max\{\text{read_TS}(X), \text{TS}(T)\}$$

Whenever the basic TO algorithm detects two *conflicting operations* that occur in the incorrect order, it rejects the later of the two operations by aborting the transaction that issued it. The schedules produced by basic TO are hence guaranteed to be *conflict serializable*. As mentioned earlier, deadlock does not occur with timestamp ordering. However, cyclic restart (and hence starvation) may occur if a transaction is continually aborted and restarted.

BTO CSS

S: $r_1(A)$ $r_2(B)$ $w_1(C)$ $r_3(B)$ $r_3(C)$ $w_2(B)$ $w_3(A)$

① $T_1 T_2 T_3$: 100, 200, 300 $\overset{100}{T_1}$ $\overset{200}{T_2}$ $\overset{300}{T_3}$

S: $\checkmark r_1(A)$ $\checkmark r_2(B)$ $\checkmark w_1(C)$ $\checkmark r_3(B)$ $\checkmark r_3(C)$ $\overset{\times}{w_2(B)}$ $w_3(A)$
 $\uparrow$

	A	B	C
RTS	100	200 300	300
WTS	0	0	100

Roll
 T_2

$\overset{100}{T_1}$ $\overset{300}{T_2}$ $\overset{200}{T_3}$: 100, 300, 200

	A	B	C
RTS	100	300	200
WTS	200	300	100

$300 > 300$
 $0 > 300$

S: $\checkmark r_1(A)$ $\checkmark r_2(B)$ $\checkmark w_1(C)$ $\checkmark r_3(B)$ $\checkmark r_3(C)$ $\checkmark w_2(B)$ $\checkmark w_3(A)$

BTO

$T_1 \rightarrow T_3 \rightarrow T_2$

100 200 300

CSS

300
 T_1

100
 T_2

200
 T_3

$S: r_1(A) \ r_2(B) \ w_1(C) \ r_3(B) \ r_3(C) \ w_2(B) \ w_3(A)$

	A	B	C
RTS	0 300	0 100 200	0
WTS	0	0	0 300

$0 > 300$

$0 > 100$

$0 > 300$

$0 > 300$

$100 > 200$

$300 > 200$

$200 > 100$

$0 > 100$

$S: \checkmark r_1(A) \ \checkmark r_2(B) \ \checkmark w_1(C) \ \checkmark r_3(B) \ \times r_3(C) \ \times w_2(B) \ \times w_3(A)$

↑

Rollback
 T_3

↑

Rollback
 T_2

Basic Timestamp Ordering (BTO)

- The timestamp-ordering protocol guarantees serializability since all the arcs in the precedence graph are of the form:

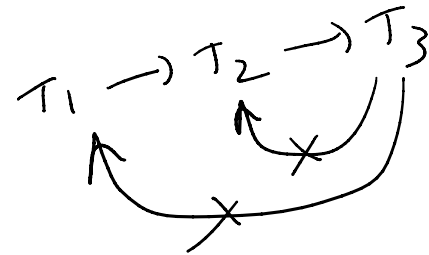


Thus, there will be no cycles in the precedence graph

- Timestamp protocol ensures freedom from deadlock as no transaction ever waits.
- But the schedule may not be cascade-free, and may not even be recoverable.

10 20 30
T₁ T₂ T₃

T₁ → T₂ → T₃

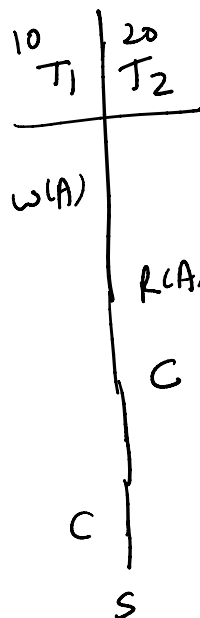


The schedules produced by basic TO are hence guaranteed to be conflict serializable.

As mentioned earlier, deadlock does not occur with timestamp ordering.

However, cyclic restart (and hence starvation) may occur if a transaction is continually aborted and restarted.

The schedule may not be cascade-free, and may not even be recoverable.



S: w₁(A) R₂(A)
✓ ✓

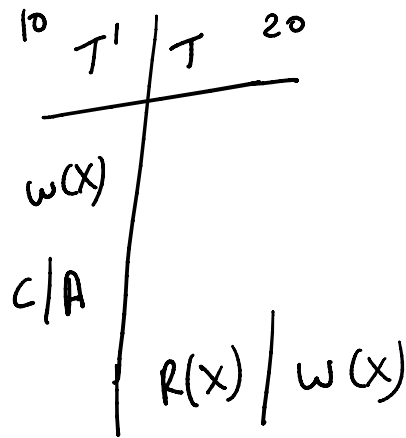
BTO

not recoverable

Strict Timestamp Ordering (STO)

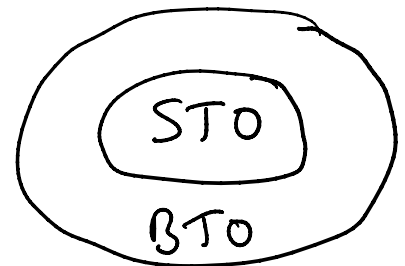
Strict Timestamp Ordering (TO). A variation of basic TO called **strict TO** ensures that the schedules are both **strict** (for easy recoverability) and (conflict) serializable. In this variation, a transaction T issues a read_item(X) or write_item(X) such that $TS(T) > \text{write_TS}(X)$ has its read or write operation delayed until the transaction T' that wrote the value of X (hence $TS(T') = \text{write_TS}(X)$) has committed or aborted.

To implement this algorithm, it is necessary to simulate the locking of an item X that has been written by transaction T' until T' is either committed or aborted. This algorithm *does not cause deadlock*, since T waits for T' only if $TS(T) > TS(T')$.



$$20 > 10$$

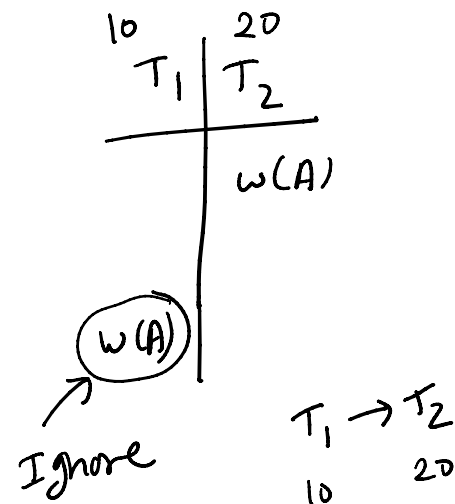
STO $\subset$ BTO



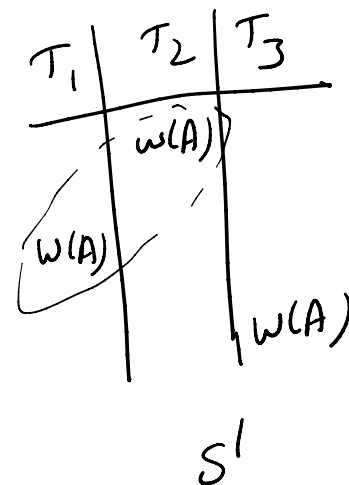
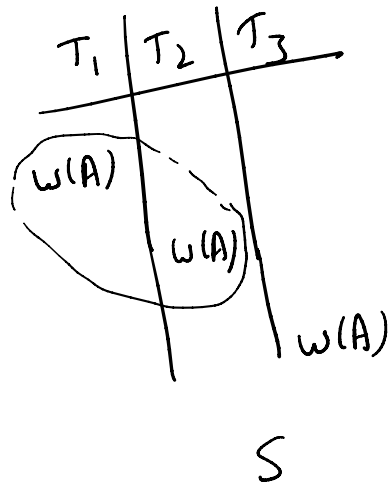
Thomas's Write Rule

Thomas' Write Rule

- Modified version of the timestamp-ordering protocol in which obsolete write operations may be ignored under certain circumstances.
- When T_i attempts to write data item Q , if $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of $\{Q\}$.
 - Rather than rolling back T_i as the timestamp ordering protocol would have done, this {write} operation can be ignored.
- Otherwise this protocol is the same as the timestamp ordering protocol.
- Thomas' Write Rule allows greater potential concurrency.
 - Allows some view-serializable schedules that are not conflict-serializable.



By ignoring the write, Thomas' write rule allows schedules that are not conflict serializable but are nevertheless correct. Those non-conflict-serializable schedules allowed satisfy the definition of view serializable schedules.

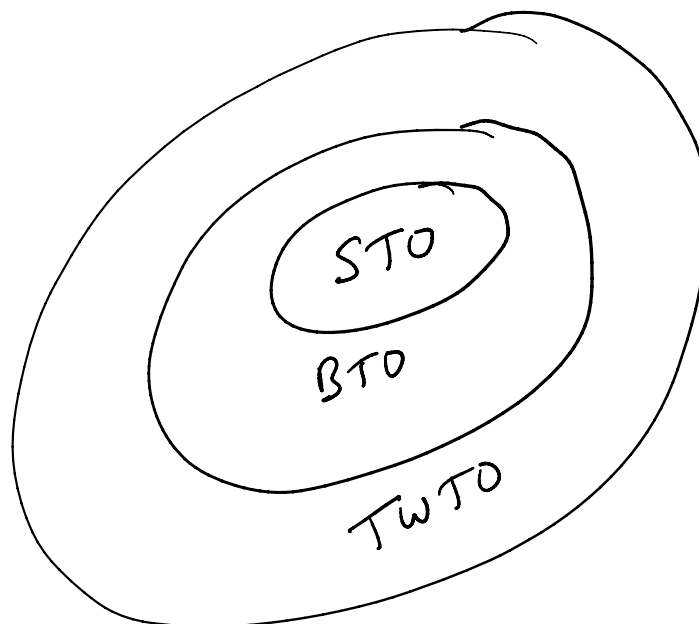


$$S \equiv S'$$

Thomas's Write Rule. A modification of the basic TO algorithm, known as **Thomas's write rule**, does not enforce conflict serializability, but it rejects fewer write operations by modifying the checks for the write_item(X) operation as follows:

- ✓ 1. If $\text{read_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation.
- ⇒ 2. If $\text{write_TS}(X) > \text{TS}(T)$, then do not execute the write operation but continue processing. This is because some transaction with timestamp greater than $\text{TS}(T)$ —and hence after T in the timestamp ordering—has already written the value of X . Thus, we must ignore the write_item(X) operation of T because it is already outdated and obsolete. Notice that any conflict arising from this situation would be detected by case (1).
- ✓ 3. If neither the condition in part (1) nor the condition in part (2) occurs, then execute the write_item(X) operation of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.

$\text{TwTO} \supset \text{BTO}$



We now consider the justification for the Thomas Write Rule. If $TS(T) < WTS(O)$, the current write action has, in effect, been made obsolete by the most recent write of O , which *follows* the current write according to the timestamp ordering on transactions. We can think of T 's write action as if it had occurred immediately *before* the most recent write of O and was never read by anyone.

If the Thomas Write Rule is not used, that is, T is aborted in case (2) above, the timestamp protocol, like 2PL, allows only conflict serializable schedules. (Both 2PL and this timestamp protocol allow schedules that the other does not.) If the Thomas Write Rule is used, some serializable schedules are permitted that are not conflict serializable, as illustrated by the schedule in Figure 19.7. Because $T2$'s write follows

$T1$	$T2$
$R(A)$	
	$W(A)$
	Commit
$W(A)$	
Commit	

Figure 19.7 A Serializable Schedule That Is Not Conflict Serializable

$T1$'s read and precedes $T1$'s write of the same object, this schedule is not conflict serializable. The Thomas Write Rule relies on the observation that $T2$'s write is never seen by any transaction and the schedule in Figure 19.7 is therefore equivalent to the serializable schedule obtained by deleting this write action, which is shown in Figure 19.8.

$T1$	$T2$
$R(A)$	
	Commit
$W(A)$	
Commit	

Figure 19.8 A Conflict Serializable Schedule